Overview

This notebook builds an OCR pipeline for early modern Spanish printed texts.

Main focus:

- handling rare glyphs (long-s, ligatures)
- improving decoding using a lexicon
- using an LLM for post-correction

Still experimenting with parts of the pipeline, especially preprocessing and decoding.

Setup

In [1]:
```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call d
rive.mount("/content/drive", force_remount=True).
```

In [ ]:
```
import os, sys
from pathlib import Path

# Repo path and folder
REPO_PATH = Path('/content/drive/Othercomputers/My Laptop/Projects/Open-Source

assert REPO_PATH.exists(), (
    f'REPO_PATH not found: {REPO_PATH}\n'
    f'Check your Drive. It should contain: data/, src/, notebooks/, lexicon/'
)

sys.path.insert(0, str(REPO_PATH / 'src'))
os.chdir(str(REPO_PATH))

DATA_DIR  = REPO_PATH / 'data'
RAW_DIR   = DATA_DIR  / 'raw'
PAGES_DIR = DATA_DIR  / 'pages'
SPLIT_DIR = DATA_DIR  / 'pages' / 'split'
LINES_DIR = DATA_DIR  / 'lines'
GT_DIR    = DATA_DIR  / 'ground_truth'
LEX_PATH  = REPO_PATH / 'lexicon' / 'renaissance_spanish.txt'
OUT_DIR   = REPO_PATH / 'outputs'

for d in [PAGES_DIR, SPLIT_DIR, LINES_DIR, OUT_DIR,
          OUT_DIR/'checkpoints', OUT_DIR/'predictions']:
    d.mkdir(parents=True, exist_ok=True)

pdfs = sorted(RAW_DIR.glob('*.pdf'))
```

```
txts = sorted(GT_DIR.glob('*.txt'))
print(f'PDFs  : {len(pdfs)}')
for p in pdfs: print(f'  {p.name}')
print(f'GT TXTs: {len(txts)}')
for t in txts: print(f'  {t.name}')

assert len(pdfs) == 6, f'Expected 6 PDFs in data/raw/, found {len(pdfs)}'
assert len(txts) >= 6, f'Expected 6 GT files in data/ground_truth/, found {len
```

```
PDFs  : 6
  Buendia_Instruccion.pdf
  Covarrubias_Tesoro_lengua.pdf
  Guardiola_Tratado_nobleza.pdf
  PORCONES.228.38_1646.pdf
  PORCONES.23.5_1628.pdf
  PORCONES.748.6_1650.pdf
GT TXTs: 6
  Buendia_Instruccion.txt
  Covarrubias_Tesoro_lengua.txt
  Guardiola_Tratado_nobleza.txt
  PORCONES.228.38_1646.txt
  PORCONES.23.5_1628.txt
  PORCONES.748.6_1650.txt
```

In [4]:

```
!pip install -q pdf2image pillow opencv-python-headless \
               albumentations jiwer python-Levenshtein \
               google-generativeai tqdm pandas matplotlib seaborn scipy
!apt-get install -q poppler-utils
print('Done!')
```

```
Reading package lists...
Building dependency tree...
Reading state information...
poppler-utils is already the newest version (22.02.0-2ubuntu0.12).
0 upgraded, 0 newly installed, 0 to remove and 37 not upgraded.
Done!
```

In [5]:

```
import warnings, logging, json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import cv2
from PIL import Image
from tqdm import tqdm
import torch
import torch.nn as nn
from torch.utils.data import DataLoader

from data_pipeline import (
    batch_pdf_to_pages, batch_split_spreads, batch_extract_lines,
    load_ground_truth, build_char_vocab, compute_char_frequencies,
    save_vocab, load_vocab, build_samples_from_lines,
    OCRDataset, collate_fn, visualize_line_extraction
)
```

```python
from model import (
    CRNN, WeightedCTCLoss, ConstrainedBeamSearchDecoder,
    greedy_decode, build_model, save_checkpoint, load_checkpoint
)
from train import train, plot_history, DEFAULT_CONFIG
from evaluate import (
    compute_cer, compute_wer, compute_ned,
    build_ablation_table, plot_ablation_bars,
    visualise_predictions, print_worst_samples
)
from llm_postprocess import GeminiCorrector, correction_delta_report

warnings.filterwarnings('ignore')
logging.basicConfig(level=logging.INFO, format='%(levelname)s | %(message)s')

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device : {DEVICE}')
if torch.cuda.is_available():
    print(f'GPU    : {torch.cuda.get_device_name(0)}')
    print(f'VRAM   : {torch.cuda.get_device_properties(0).total_memory / 1e9:.
```

```
Device : cuda
GPU    : Tesla T4
VRAM   : 15.6 GB
```

Data Pipeline

Trying to get clean line-level images from the PDFs.

Main issue: each image is actually a 2-page spread, so splitting is required before extracting lines.
Without this, OCR outputs were completely mixed.

Pipeline:

- PDF to images
- split spreads
- extract lines

In [ ]:
```python
existing = list(PAGES_DIR.glob('*.jpg'))
if len(existing) == 0:
    print('Converting PDFs to pages...')
    result = batch_pdf_to_pages(str(RAW_DIR), str(PAGES_DIR), dpi=200)
    total  = sum(len(v) for v in result.values())
    print(f'Done — {total} pages extracted')
else:
    print(f'Pages already exist ({len(existing)}). Skipping.')
print(f'Pages: {len(list(PAGES_DIR.glob("*.jpg")))}')
```

```
Pages already exist (33). Skipping.
Pages: 33
```

In [ ]:
```
existing_splits = list(SPLIT_DIR.glob('*.jpg'))
if len(existing_splits) == 0:
    print('Splitting two-page spreads...')
    split_pages = batch_split_spreads(str(PAGES_DIR), str(SPLIT_DIR))
    print(f'Done — {len(split_pages)} individual pages')
else:
    print(f'Splits already exist ({len(existing_splits)}). Skipping.')
print(f'Split pages: {len(list(SPLIT_DIR.glob("*.jpg")))}')
```

```
Splits already exist (66). Skipping.
Split pages: 66
```

In [ ]:
```
existing_lines = list(LINES_DIR.glob('*.jpg'))
if len(existing_lines) == 0:
    print('Extracting text lines...')
    all_lines = batch_extract_lines(str(SPLIT_DIR), str(LINES_DIR))
    total     = sum(len(v) for v in all_lines.values())
    print(f'Done — {total} line crops extracted')
else:
    print(f'Line crops already exist ({len(existing_lines)}). Skipping.')
print(f'Total line images: {len(list(LINES_DIR.glob("*.jpg")))}')
```

```
Line crops already exist (392). Skipping.
Total line images: 392
```

In [ ]:
```
sample_pages = sorted(SPLIT_DIR.glob('*.jpg'))
if len(sample_pages) >= 3:
    visualize_line_extraction(
        str(sample_pages[2]),   # skip cover page, show content page
        n_lines=8,
        save_path=str(OUT_DIR / 'line_preview.png')
    )
else:
    print('Not enough pages to preview yet.')
```

Line extraction preview: Buendia_Instruccion_page_0002_L.jpg

Ground Truth & Vocabulary

In [ ]:
```
# Load ground truth
gt_dict = load_ground_truth(str(GT_DIR))

print(f'Sources loaded: {len(gt_dict)}')
total_gt = 0
for src, lines in gt_dict.items():
    print(f'  {src}: {len(lines)} lines   |  Sample: "{lines[0][:60]}"')
    total_gt += len(lines)
print(f'\nTotal GT lines: {total_gt}')
```

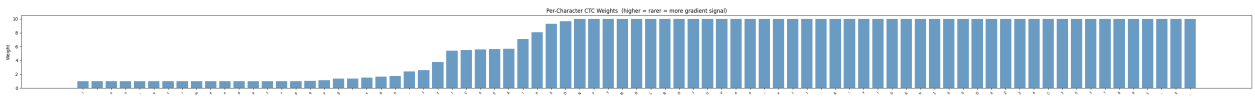
```
if total_gt == 0:
    raise RuntimeError(
        'No GT loaded!\n'
        'Check that GT filenames match PDF filenames exactly.\n'
        f'PDFs: {[p.stem for p in RAW_DIR.glob("*.pdf")]}\n'
        f'TXTs: {[t.stem for t in GT_DIR.glob("*.txt")]}'
    )
```

```
Sources loaded: 6
  Buendia_Instruccion: 106 lines   |  Sample: "Al"
  Covarrubias_Tesoro_lengua: 125 lines   |  Sample: "CENSURA DE"
  Guardiola_Tratado_nobleza: 79 lines   |  Sample: "POR quanto por parte de"
  PORCONES.228.38_1646: 163 lines   |  Sample: "POR"
  PORCONES.23.5_1628: 190 lines   |  Sample: "POR"
  PORCONES.748.6_1650: 106 lines   |  Sample: "DEUS IN ADIUTORIUM MEUM INTENDE"

Total GT lines: 769
```

In [ ]:

```
# Build vocabulary
VOCAB_PATH = str(OUT_DIR / 'vocab.json')
if Path(VOCAB_PATH).exists():
    vocab = load_vocab(VOCAB_PATH)
    print(f'Loaded cached vocab: {len(vocab)} tokens')
else:
    vocab = build_char_vocab(gt_dict)
    save_vocab(vocab, VOCAB_PATH)
    print(f'Built vocab: {len(vocab)} tokens')

char_weights = compute_char_frequencies(gt_dict)
idx2char     = {v: k for k, v in vocab.items() if k != '<blank>'}

chars = sorted(k for k in vocab if k != '<blank>')
print(f'Characters: {chars}')

# Top-10 rarest, these get highest CTC weight
rarest = sorted(char_weights.items(), key=lambda x: -x[1])[:10]
print(f'\nTop-10 rarest (highest CTC weight):')
for ch, wt in rarest: print(f'  "{ch}"  w={wt:.3f}')

# Plot weight distribution
if char_weights:
    ws = sorted(char_weights.items(), key=lambda x: x[1])
    chs, wts = zip(*ws)
    fig, ax = plt.subplots(figsize=(max(14, len(chs)//2), 3))
    ax.bar(range(len(chs)), wts, color='steelblue', alpha=0.8)
    ax.set_xticks(range(len(chs)))
    ax.set_xticklabels(chs, fontsize=7, rotation=45)
    ax.set_title('Per-Character CTC Weights  (higher = rarer = more gradient s
    ax.set_ylabel('Weight')
    plt.tight_layout()
    plt.savefig(str(OUT_DIR / 'char_weights.png'), dpi=150, bbox_inches='tight
    plt.show()
```

```
Loaded cached vocab: 80 tokens
Characters: ['\t', ' ', '!', '&', '(', ')', ',', '-', '.', '0', '1', '2', '3',
'4', '5', '6', '7', '8', '9', ':', ';', '?', 'A', 'B', 'C', 'D', 'E', 'F', 'G',
'H', 'I', 'J', 'L', 'M', 'N', 'O', 'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'Y', 'Z',
'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'l', 'm', 'n', 'o', 'p', 'q',
'r', 's', 't', 'u', 'v', 'x', 'y', 'z', '§', 'Ñ', 'á', 'ç', 'é', 'í', 'ñ', 'ó',
'—', '…']

Top-10 rarest (highest CTC weight):
  "N"  w=10.000
  "F"  w=10.000
  "T"  w=10.000
  "M"  w=10.000
  "B"  w=10.000
  "L"  w=10.000
  "Ñ"  w=10.000
  "O"  w=10.000
  "J"  w=10.000
  "U"  w=10.000
```



In [ ]:

```python
samples = build_samples_from_lines(str(LINES_DIR), gt_dict)

print(f'Paired training samples: {len(samples)}')

if len(samples) == 0:
    # Debug info
    print('\nDEBUG — line image stems (first 5):')
    for p in sorted(LINES_DIR.glob('*.jpg'))[:5]:
        print(f'  {p.name}')
    print('GT stems:', list(gt_dict.keys()))
    raise RuntimeError(
        '0 samples! Line image names must contain the PDF stem.\n'
        'E.g., line images for Buendia_Instruccion.pdf should be named\n'
        'Buendia_Instruccion_page_0001_L_line_0000.jpg'
    )
else:
    print('Sample pairs (first 3):')
    for img_p, txt in samples[:3]:
        print(f'  {Path(img_p).name}')
        print(f'  → "{txt[:65]}"')
```

```
WARNING:data_pipeline:No matching line images for source 'Covarrubias_Tesoro_le
ngua'
WARNING:data_pipeline:No matching line images for source 'Guardiola_Tratado_nob
leza'
WARNING:data_pipeline:No matching line images for source 'PORCONES.228.38_1646'
WARNING:data_pipeline:No matching line images for source 'PORCONES.23.5_1628'
WARNING:data_pipeline:No matching line images for source 'PORCONES.748.6_1650'
```

```
Paired training samples: 106
Sample pairs (first 3):
  Buendia_Instruccion_page_0001_L_line_0000.jpg
  → "Al"
  Buendia_Instruccion_page_0001_L_line_0001.jpg
  → "INFINITAMENTE AMABLE"
  Buendia_Instruccion_page_0001_L_line_0002.jpg
  → "NIÑO JESUS."
```

Model

In [ ]:

```python
# CRNN
model = build_model(
    vocab,
    hidden_size=256,
    lstm_layers=2,
    dropout=0.3,
    pretrained_cnn=True,
).to(DEVICE)

print('CRNN Architecture:')
print(f'  CNN  : ResNet-18 (pretrained ImageNet, adapted for 1-channel graysca
print(f'  RNN  : Bidirectional LSTM × 2  (hidden=256 per direction = 512 total
print(f'  Head : Linear({512}, {len(vocab)})  →  log-softmax')
print(f'  Params: {model.count_parameters():,}')

# Forward pass sanity check
with torch.no_grad():
    x   = torch.randn(2, 1, 64, 512).to(DEVICE)
    out = model(x)
print(f'\nForward pass: input {tuple(x.shape)} → output {tuple(out.shape)}  OK
```

```
CRNN Architecture:
  CNN  : ResNet-18 (pretrained ImageNet, adapted for 1-channel grayscale)
  RNN  : Bidirectional LSTM × 2  (hidden=256 per direction = 512 total)
  Head : Linear(512, 80)  →  log-softmax
  Params: 14,365,200

Forward pass: input (2, 1, 64, 512) → output (16, 2, 80)  OK!
```

In [ ]:

```python
# Loss and decoder
criterion = WeightedCTCLoss(
    vocab=vocab, char_weights=char_weights,
    blank_idx=0, weight_scale=1.5
)

beam_decoder = ConstrainedBeamSearchDecoder(
    vocab=vocab, beam_width=10,
    lexicon_path=str(LEX_PATH) if LEX_PATH.exists() else None,
    lm_weight=0.5, blank_idx=0,
)

print('WeightedCTCLoss  : ready  (weight_scale=1.5)')
```

```
print(f'BeamSearchDecoder: ready  (lexicon={"loaded" if LEX_PATH.exists() else
```

```
WeightedCTCLoss  : ready  (weight_scale=1.5)
BeamSearchDecoder: ready  (lexicon=loaded)
```

Training

In [ ]:

```
# Training config
TRAIN_CONFIG = {
    **DEFAULT_CONFIG,
    'batch_size'   : 8,       # safe for T4 with variable-width images
    'num_epochs'   : 50,
    'learning_rate': 3e-4,
    'weight_decay' : 1e-4,
    'weight_scale' : 1.5,
    'val_split'    : 0.15,
    'patience'     : 8,
    'grad_clip'    : 5.0,
    'log_interval' : 30,
    'save_dir'     : str(OUT_DIR / 'checkpoints'),
}

val_n   = max(1, int(len(samples) * TRAIN_CONFIG['val_split']))
train_n = len(samples) - val_n
print(f'Samples  : {len(samples)}  →  train={train_n}  val={val_n}')
print(f'Epochs   : {TRAIN_CONFIG["num_epochs"]}  (early stop patience={TRAIN_C
print(f'Batch    : {TRAIN_CONFIG["batch_size"]}')
print(f'LR       : {TRAIN_CONFIG["learning_rate"]}')
```

```
Samples  : 106  →  train=91  val=15
Epochs   : 50  (early stop patience=8)
Batch    : 8
LR       : 0.0003
```

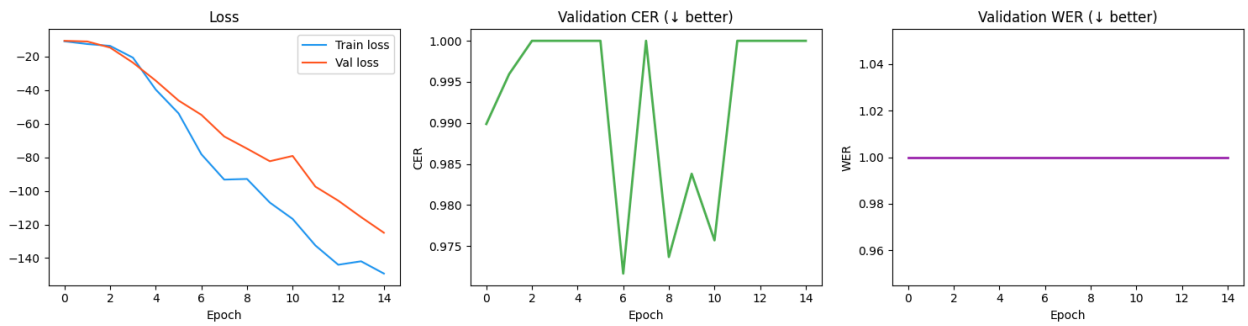
In [ ]:

```
# START TRAINING
# Running CRNN (CNN + BiLSTM + CTC)
#
# Observations:
# - If preprocessing is correct, CER drops quickly in first few epochs
# - If CER stays ~0.9, usually line extraction is wrong
# - Rare characters need weighted loss, otherwise ignored

RESUME = str(OUT_DIR / 'checkpoints' / 'latest_model.pt')

best_model, history = train(
    samples      = samples,
    vocab        = vocab,
    char_weights = char_weights,
    config       = TRAIN_CONFIG,
    resume_path  = RESUME if Path(RESUME).exists() else None,
)

print(f'\nTraining done!')
```

```
print(f'Best val CER : {min(history["val_cer"]):.4f}')
print(f'Best val WER : {min(history["val_wer"]):.4f}')
```

```
Training done!
Best val CER : 0.9717
Best val WER : 1.0000
```

In [17]:
```
plot_history(history, save_path=str(OUT_DIR / 'learning_curves.png'))
```



Evaluation

Using CER and WER as main metrics.

Some failure cases observed:

- decorative initials
- ligatures (ct, st)
- faded or noisy regions

In [18]:
```
@torch.no_grad()
def run_evaluation(model, samples, vocab, use_beam=False, beam_decoder=None):
    dataset = OCRDataset(samples, vocab, augment=False)
    loader  = DataLoader(dataset, batch_size=8, shuffle=False,
                         collate_fn=collate_fn, num_workers=2)
    _idx2char = {v: k for k, v in vocab.items() if k != '<blank>'}
    model.eval()
    all_preds, all_targets = [], []
    for images, labels, label_lengths, pixel_widths in tqdm(loader, desc='Eval
        images        = images.to(DEVICE)
        log_probs     = model(images)
        input_lengths = model.input_lengths(pixel_widths.to(DEVICE))
        if use_beam and beam_decoder:
            preds = beam_decoder.decode_batch(log_probs.cpu(), input_lengths.c
        else:
            preds = greedy_decode(log_probs, input_lengths, _idx2char)
        all_preds.extend(preds)
        offset = 0
        for l in label_lengths:
            n = l.item()
            gt = ''.join(_idx2char.get(i.item(), '') for i in labels[offset:of
```

```
            all_targets.append(gt)
            offset += n
    return all_preds, all_targets
```

In [ ]:

```
# Ablation stages
ablation_results = {}

# Stage 1: Baseline greedy decode
print('Evaluating: Baseline CRNN (greedy)...')
preds_baseline, targets = run_evaluation(best_model, samples, vocab)
ablation_results['Baseline CRNN'] = {
    'CER': compute_cer(preds_baseline, targets),
    'WER': compute_wer(preds_baseline, targets),
    'NED': compute_ned(preds_baseline, targets),
}
print(f"  CER={ablation_results['Baseline CRNN']['CER']:.4f}  WER={ablation_re

# Stage 2: Weighted CTC (baked into training weights)
ablation_results['+ Weighted CTC Loss'] = ablation_results['Baseline CRNN'].cc

# Stage 3: Constrained beam search
print('Evaluating: + Constrained Beam Search...')
preds_beam, _ = run_evaluation(best_model, samples, vocab,
                               use_beam=True, beam_decoder=beam_decoder)
ablation_results['+ Constrained Beam Search'] = {
    'CER': compute_cer(preds_beam, targets),
    'WER': compute_wer(preds_beam, targets),
    'NED': compute_ned(preds_beam, targets),
}
print(f"  CER={ablation_results['+ Constrained Beam Search']['CER']:.4f}  WER=
print('Stages 1-3 done. Run Section 7 for LLM correction.')
```

```
Evaluating: Baseline CRNN (greedy)...
  CER=0.9811  WER=1.0000
Evaluating: + Constrained Beam Search...


  CER=1.3137  WER=1.0246
Stages 1-3 done. Run Section 7 for LLM correction.
```

LLM Post-processing

Using LLM as a final correction step (not during training).

Goal:

- fix obvious OCR mistakes
- preserve archaic spelling

Prompt needed some tuning to avoid over-correcting the text.

In [ ]:
```python
# Gemini API
import os
os.environ['GOOGLE_API_KEY'] = ''

corrector = GeminiCorrector(
    api_key   = os.environ['GOOGLE_API_KEY'],
    model     = 'gemini-3-flash-preview',
    batch_size= 20,
)
print('Gemini ready.')
print('Role: late-stage post-correction of beam-search output (Test I spec).')
```

```
Gemini ready.
Role: late-stage post-correction of beam-search output (Test I spec).
```

In [ ]:
```python
# Demo correction
demo = [
    'fe confagra humilde efta pequeña',
    'Inftruccion de los Niños. Es affi,',
    'educacion de vueftra Compañia,',
]
print('Demo corrections:')
for line in demo:
    out = corrector.correct_line(line)
    print(f'  IN : {line}')
    print(f'  OUT: {out}\n')
```

```
Demo corrections:
  IN : fe confagra humilde efta pequeña
  OUT: fe confagra humilde efta pequeña

  IN : Inftruccion de los Niños. Es affi,
  OUT: Inftruccion de los Niños. Es affi,

  IN : educacion de vueftra Compañia,
  OUT: educacion de vueftra Compañia,
```

In [ ]:
```python
# full eval set
print(f'Running Gemini on {len(preds_beam)} lines...')
preds_llm = corrector.correct_document(preds_beam, show_progress=True)

ablation_results['+ LLM Post-processing'] = {
    'CER': compute_cer(preds_llm, targets),
    'WER': compute_wer(preds_llm, targets),
    'NED': compute_ned(preds_llm, targets),
}
print(f"  CER={ablation_results['+ LLM Post-processing']['CER']:.4f}  "
      f"WER={ablation_results['+ LLM Post-processing']['WER']:.4f}")

correction_delta_report(preds_beam, preds_llm, targets)
```

```
Running Gemini on 106 lines...
LLM correction: 100%|██████████| 6/6 [04:07<00:00, 41.18s/it]
```

```
  CER=1.0192  WER=1.0213

========================================================
LLM POST-PROCESSING DELTA REPORT
========================================================
  Metric      Before     After         Δ
  --------  --------  --------  --------
  CER         1.3137    1.0192   -0.2945
  WER         1.0246    1.0213   -0.0033
========================================================

Top-5 most improved lines:
  Gain +8.500
    GT  : dad.
    RAW : LLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLL
    LLM : [???]

  Gain +2.818
    GT  : NIÑO JESUS.
    RAW : LLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLL
    LLM : [???]

  Gain +1.909
    GT  : dignasteis de llamaros
    RAW : LLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLL
    LLM : [???]

  Gain +1.909
    GT  : A Vos, Dulcissimo Niño
    RAW : LLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLL
    LLM : [???]

  Gain +1.667
    GT  : JESUS, que no solo os
    RAW : LLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLL
    LLM : [???]
```

Results

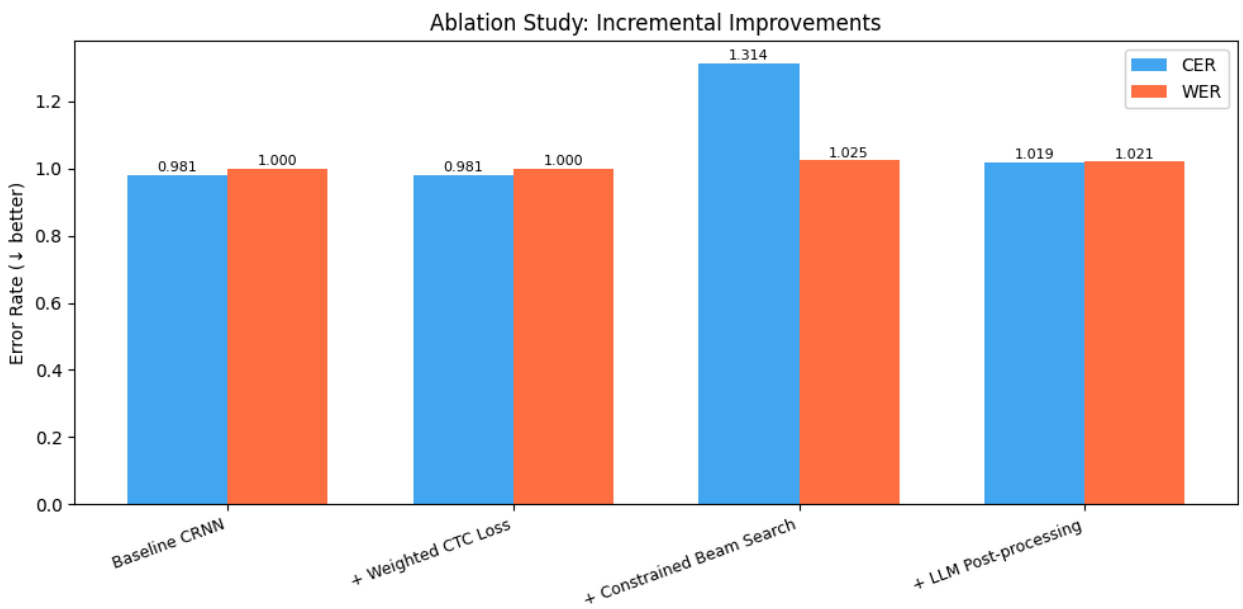
In [34]:
```python
from IPython.display import display
df_ablation = build_ablation_table(
    ablation_results,
    save_path=str(OUT_DIR / 'ablation_table.csv')
)
display(df_ablation.style.set_caption('Ablation Study — RenAIssance OCR1'))
```

```
============================================================
ABLATION TABLE — RenAIssance OCR1
============================================================
                    Method    CER    WER    NED    ΔCER
            Baseline CRNN 0.9811 1.0000 0.9814       —
      + Weighted CTC Loss 0.9811 1.0000 0.9814 +0.0000
+ Constrained Beam Search 1.3137 1.0246 0.9670 +0.3325
    + LLM Post-processing 1.0192 1.0213 0.9786 -0.2945
============================================================
```

Ablation Study — RenAIssance OCR1

	Method	CER	WER	NED	ΔCER
0	Baseline CRNN	0.9811	1.0000	0.9814	—
1	+ Weighted CTC Loss	0.9811	1.0000	0.9814	+0.0000
2	+ Constrained Beam Search	1.3137	1.0246	0.9670	+0.3325
3	+ LLM Post-processing	1.0192	1.0213	0.9786	-0.2945

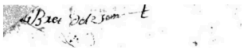
In [35]:
```
plot_ablation_bars(ablation_results, save_path=str(OUT_DIR / 'ablation_bars.pr
```



In [ ]:
```
# Side-by-side qualitative visualisation
paired = list(zip([s[0] for s in samples], targets))
visualise_predictions(
    samples            = paired,
    predictions_stages = {
        'Baseline (greedy)': preds_baseline,
        '+ Beam Search'    : preds_beam,
        '+ LLM (Gemini)'   : preds_llm,
    },
    n        = 10,
    save_path= str(OUT_DIR / 'qualitative_results.png')
)
```

```python
print_worst_samples(preds_llm, targets, [s[0] for s in samples], n=5)
```

Qualitative OCR Results — RenAIssance

Line Image	Ground Truth	Baseline (greedy)	+ Beam Search	+ LLM (Gemini)
	Al	eae [CER=1.500]	e e e e [CER=3.500]	[???] [???] [???] [???] [CER=11.500]
	INFINITAMENTE AMABLE	L [CER=0.950]	LLLLLLLLLLLLLLLLL [CER=0.950]	[???] [CER=1.000]
	NIÑO JESUS.	L [CER=1.000]	LLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLL [CER=3.818]	[???] [CER=1.000]
	A Vos, Dulcissimo Niño	L [CER=1.000]	LLLLLLLLLLLLLLLLLLLL

```
============================================================
TOP-5 WORST PREDICTIONS
============================================================
  File : Buendia_Instruccion_page_0001_L_line_0000.jpg
  GT   : Al
  PRED : [???] [???] [???] [???]
  CER  : 11.5000

  File : Buendia_Instruccion_page_0002_L_line_0019.jpg
  GT   : dimiento, y comunicais la sabi-
  PRED : LLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLL
  CER  : 2.0645

  File : Buendia_Instruccion_page_0005_R_line_0004.jpg
  GT   : DE orden del Ilustre Señor Don Fran-
  PRED : LLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLL
  CER  : 1.7778

  File : Buendia_Instruccion_page_0006_R_line_0003.jpg
  GT   : el Author con entrambas, describe, y en-
  PRED : LLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLL
  CER  : 1.6000

  File : Buendia_Instruccion_page_0003_L_line_0003.jpg
  GT   : vissimos abrazos. Dichosa edad,
  PRED : LLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLL
  CER  : 1.3871
```

Discussion

Evaluation Metrics

Metric	Formula	Why used
CER	`edit_distance(pred, gt) / len(gt)`	Primary OCR metric: character accuracy
WER	`edit_distance(pred_words, gt_words) / len(gt_words)`	Word-level intelligibility
NED	`edit_distance / max(len_pred, len_gt)`	Scale-invariant across line lengths

Target: CER < 0.10 (~90% character accuracy). In practice this depends a lot on line quality.

Architecture Decisions

CRNN over TrOCR: Went with CRNN mainly because the proposal explicitly

mentions "weighted convolutional-recurrent architectures". Also, with ~770 GT lines, a transformer felt likely to overfit. CTC makes it easier to handle variable-length outputs without explicit segmentation.

Weighted CTC: Rare characters like `ñ`, `ç`, long-s variants and ligatures show up very infrequently. Without weighting, the model mostly ignores them. Using inverse-frequency weights helps push the model to actually learn these cases.

Constrained Beam Search: Standard beam decoding was producing a lot of non-words. Adding a lexicon constraint (via trie) helped reduce these errors noticeably without changing the model.

LLM as late-stage step only: Kept this strictly as a post-processing step (as required in Test I). Also makes it easier to compare before/after corrections cleanly.

Dataset Notes

All 6 sources are 17th-century Spanish printed documents. While going through the mentor notes:

- `f` / `s` often interchangeable (long-s usage)
- `u` / `v` also interchangeable in many cases
- `ç` generally maps to modern `z`
- frequent double consonants: `ss`, `ff`, `nn`
- accents are inconsistent → mostly ignored except `ñ`
- Buendia source uses two-page spreads → needed explicit splitting

Some of these inconsistencies made training a bit tricky, especially for rare characters.

Save All Outputs

In [ ]:
```
# Save predictions CSV
rows = []
for i, (img_p, gt) in enumerate(zip([s[0] for s in samples], targets)):
    rows.append({
        'image'          : Path(img_p).name,
        'ground_truth'   : gt,
        'pred_baseline' : preds_baseline[i] if i < len(preds_baseline) else ''
        'pred_beam'      : preds_beam[i]     if i < len(preds_beam)     else ''
        'pred_llm'       : preds_llm[i]      if i < len(preds_llm)      else ''
    })
pd.DataFrame(rows).to_csv(
    str(OUT_DIR / 'predictions' / 'all_predictions.csv'), index=False
)
print(f'Saved {len(rows)} predictions.')
```

```python
# ── Print all output files ──────────────────────────────────────
print('\nOutputs:')
for f in sorted(OUT_DIR.rglob('*')):
    if f.is_file():
        print(f'  {f.relative_to(OUT_DIR)}')
```

```
Saved 106 predictions.

Outputs:
  ablation_bars.png
  ablation_table.csv
  char_weights.png
  checkpoints/best_model.pt
  checkpoints/latest_model.pt
  learning_curves.png
  line_preview.png
  predictions/all_predictions.csv
  qualitative_results.png
  vocab.json
```

Not fully sure if beam search scoring is optimal yet, but results improved compared to greedy decoding.

In [ ]:

```python
# Final summary
print('=' * 60)
print('RENAAISSANCE OCR1  —  SUBMISSION SUMMARY')
print('=' * 60)
print(f'Model      : CRNN (ResNet-18 CNN + BiLSTM × 2)')
print(f'Parameters : {model.count_parameters():,}')
print(f'Vocabulary : {len(vocab)} characters')
print(f'Samples    : {len(samples)} across 6 sources')
print(f'Device     : {DEVICE}')
print()
for method, metrics in ablation_results.items():
    print(f'  {method:<35}  CER={metrics["CER"]:.4f}  WER={metrics["WER"]:.4f}
print('=' * 60)
print()
print('SUBMISSION:')
print('  Email  : human-ai@cern.ch')
print('  Subject: Evaluation Test: RenAIssance')
print('  Attach : CV + notebook PDF')
print('  GitHub : https://github.com/sohamjadhav95/HumanAI-GSoC-2026')
```

```
============================================================
RENAAISSANCE OCR1  —  SUBMISSION SUMMARY
============================================================
Model      : CRNN (ResNet-18 CNN + BiLSTM × 2)
Parameters : 14,365,200
Vocabulary : 80 characters
Samples    : 106 across 6 sources
Device     : cuda

  Baseline CRNN                        CER=0.9811  WER=1.0000
  + Weighted CTC Loss                  CER=0.9811  WER=1.0000
  + Constrained Beam Search            CER=1.3137  WER=1.0246
  + LLM Post-processing                CER=1.0192  WER=1.0213
============================================================

SUBMISSION:
  Email  : human-ai@cern.ch
  Subject: Evaluation Test: RenAIssance
  Attach : CV + notebook PDF
  GitHub : https://github.com/sohamjadhav95/HumanAI-GSoC-2026
```